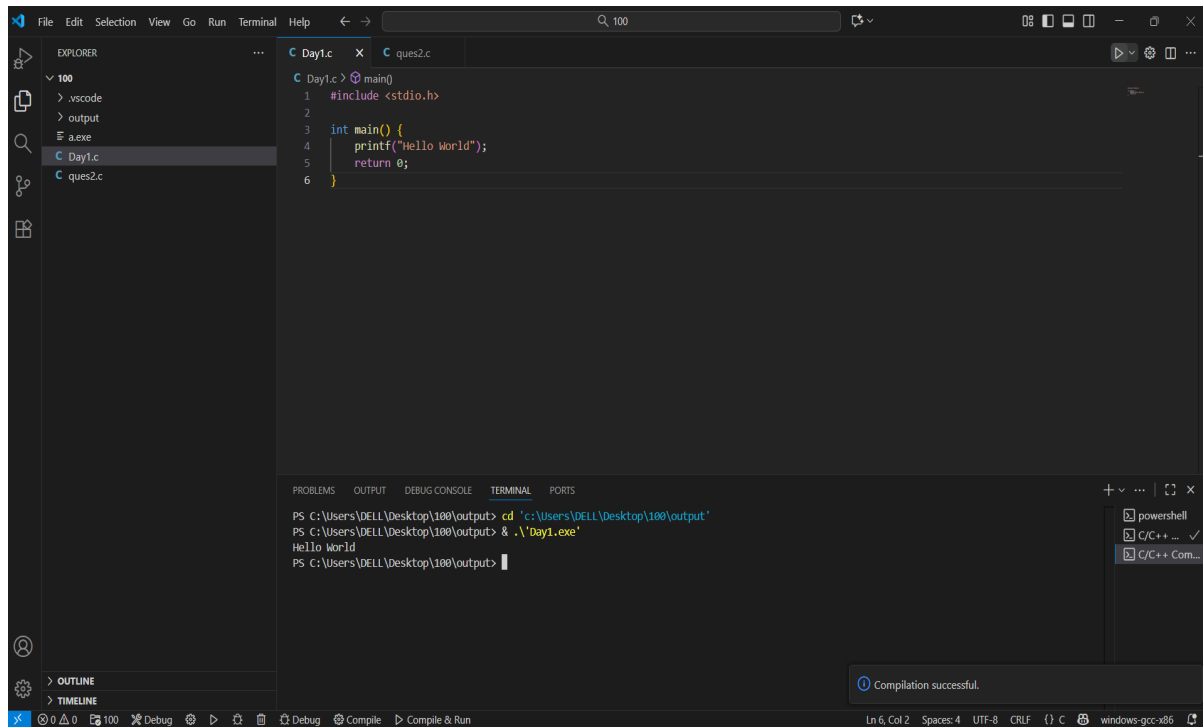
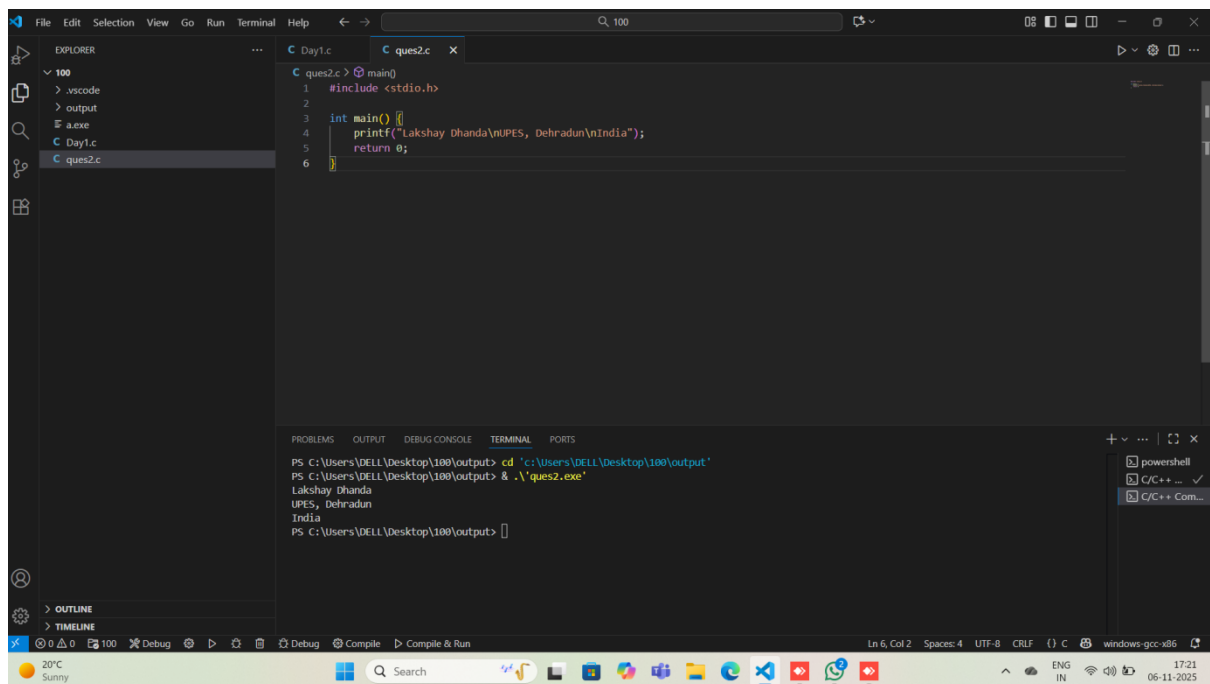


Experiment 1: Installation, Environment Setup and starting with C language

1. Write a C program to print " Hello World"



2. Write a C Program to print the address in multiple lines (new line).

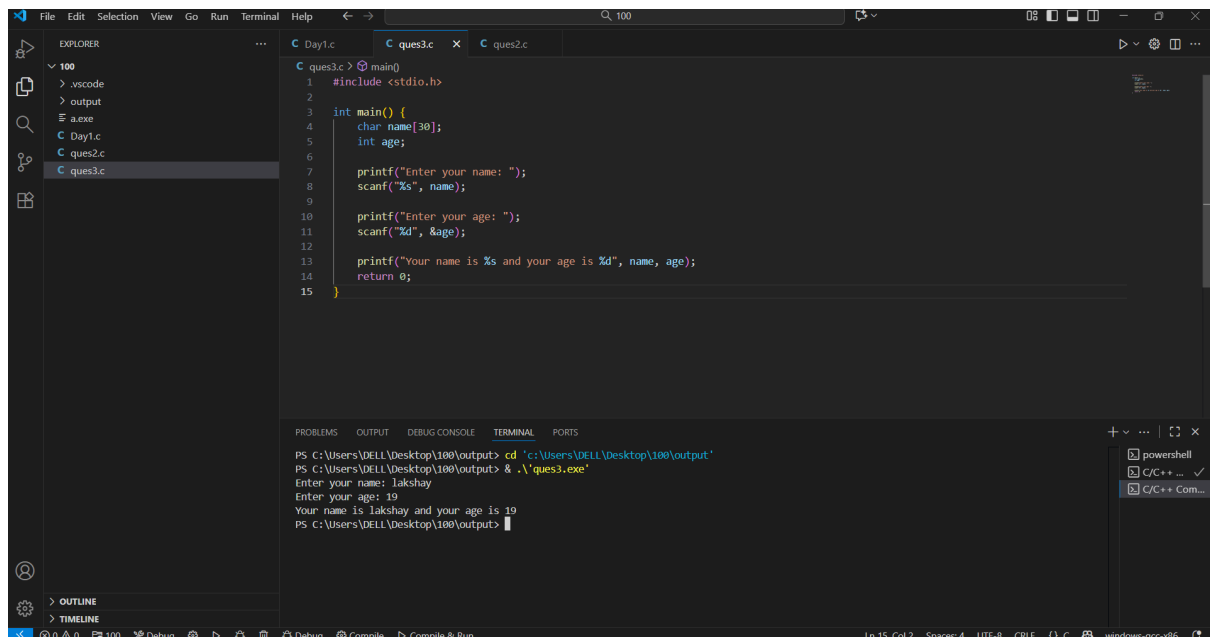


The screenshot shows the Visual Studio Code editor with a C program named `ques2.c` open. The program includes `<stdio.h>` and uses `printf` to print the address "Lakshay Dhanda" and "UPES, Dehradun" on separate lines. The terminal output shows the program being compiled and run, resulting in the expected output.

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Lakshay Dhanda\nUPES, Dehradun\nIndia");
5     return 0;
6 }
```

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques2.exe
Lakshay Dhanda
UPES, Dehradun
India
PS C:\Users\DELL\Desktop\100\output>
```

3. Write a program that prompts the user to enter their name and age.

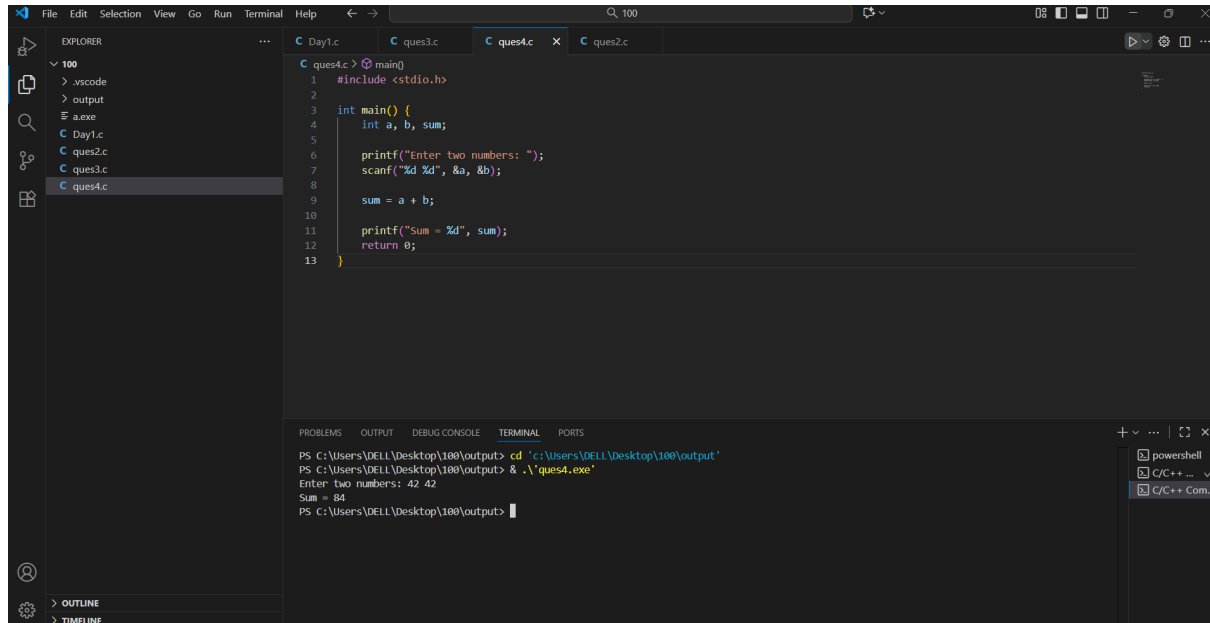


The screenshot shows the Visual Studio Code editor with a C program named `ques3.c` open. The program includes `<stdio.h>` and uses `printf` and `scanf` to prompt the user for their name and age, and then prints the entered information. The terminal output shows the program being compiled and run, with the user input "Lakshay" and "19" being displayed.

```
1 #include <stdio.h>
2
3 int main() {
4     char name[30];
5     int age;
6
7     printf("Enter your name: ");
8     scanf("%s", name);
9
10    printf("Enter your age: ");
11    scanf("%d", &age);
12
13    printf("Your name is %s and your age is %d", name, age);
14    return 0;
15 }
```

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques3.exe
Enter your name: Lakshay
Enter your age: 19
Your name is Lakshay and your age is 19
PS C:\Users\DELL\Desktop\100\output>
```

4. Write a C program to add two numbers, take number from user.



The screenshot displays the Visual Studio Code interface. The Explorer sidebar on the left shows a project structure with files like .vscode, output, a.exe, and several C source files (Day1.c, ques2.c, ques3.c, and ques4.c). The main editor window is open to 'ques4.c', which contains the following C code:

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b, sum;
5
6     printf("Enter two numbers: ");
7     scanf("%d %d", &a, &b);
8
9     sum = a + b;
10
11    printf("Sum = %d", sum);
12    return 0;
13 }
```

Below the editor, the TERMINAL panel shows the execution of the program. The commands and output are as follows:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques4.exe
Enter two numbers: 42 42
Sum = 84
PS C:\Users\DELL\Desktop\100\output>
```

EXPERIMENT 2: Operator

1.WAP a C program to calculate the area and perimeter of a rectangle based on its length and width.

The screenshot shows the Visual Studio Code editor with a C program named `ques5.c` open. The program calculates the area and perimeter of a rectangle. The Explorer sidebar on the left shows a project structure with folders `.vscode` and `output`, and files `a.exe`, `Day1.c`, `ques2.c`, `ques3.c`, `ques4.c`, and `ques5.c`. The main editor displays the following code:

```
1 #include <stdio.h>
2
3 int main() {
4     float length, width, area, perimeter;
5
6     printf("Enter length and width: ");
7     scanf("%f %f", &length, &width);
8
9     area = length * width;
10    perimeter = 2 * (length + width);
11
12    printf("Area = %.2f\n", area);
13    printf("Perimeter = %.2f", perimeter);
14
15    return 0;
16 }
```

The bottom panel shows the TERMINAL output:

```
PS C:\Users\DELL\Desktop\100> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques5.exe
Enter length and width: 7 8
Area = 56.00
Perimeter = 30.00
PS C:\Users\DELL\Desktop\100\output>
```

2. WAP a C program to Convert temperature from Celsius to Fahrenheit using the formula: $F = (C * 9/5) + 32$.

The screenshot shows the Visual Studio Code editor with a C program named `ques6.c` open. The program converts temperature from Celsius to Fahrenheit. The Explorer sidebar on the left shows the same project structure as the previous screenshot, with `ques6.c` now selected. The main editor displays the following code:

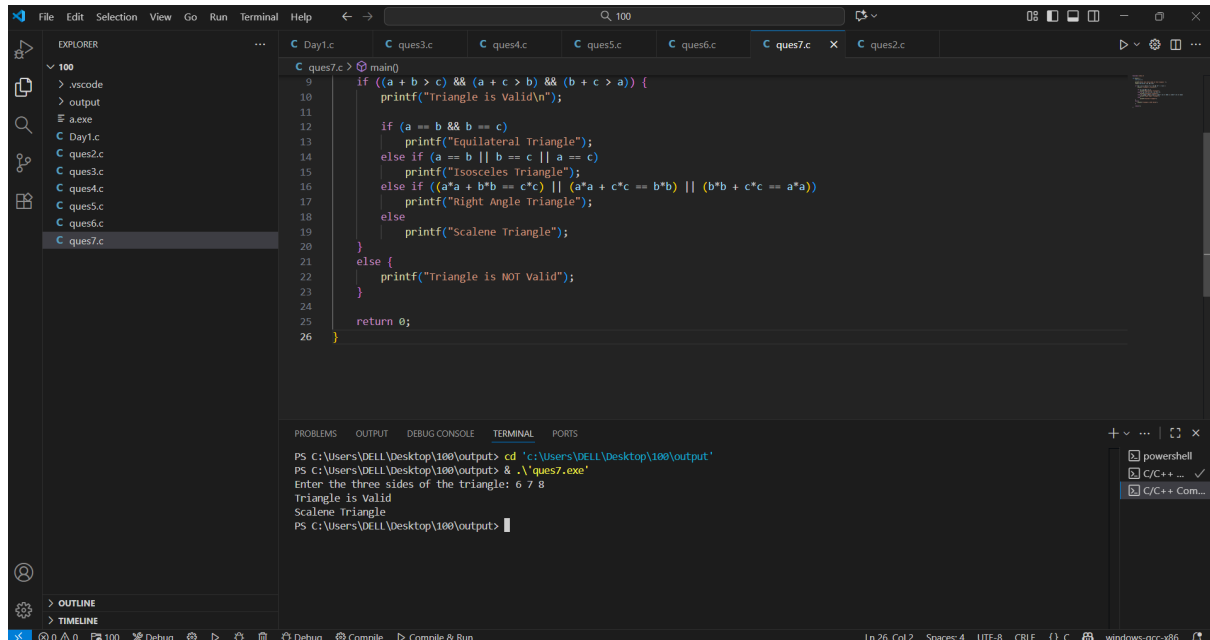
```
2
3 int main() {
4     float C, F;
5
6     printf("Enter temperature in Celsius: ");
7     scanf("%f", &C);
8
9     F = (C * 9 / 5) + 32;
10
11    printf("Temperature in Fahrenheit = %.2f", F);
12
13    return 0;
14 }
```

The bottom panel shows the TERMINAL output:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques6.exe
Enter temperature in Celsius: 8
Temperature in Fahrenheit = 46.40
PS C:\Users\DELL\Desktop\100\output>
```

EXPERIMENT 3.1: Conditional Statement

1. WAP to take check if the triangle is valid or not. If the validity is established, do check if the triangle is isosceles, equilateral, right angle, or scalene. Take sides of the triangle as input from a user.



The screenshot shows a C++ IDE with a file explorer on the left containing files like .vscode, output, a.exe, and several .c files. The main editor displays a C++ program in 'ques7.c' that checks if three sides (a, b, c) form a valid triangle. If valid, it further categorizes the triangle as equilateral, isosceles, right-angled, or scalene. The terminal at the bottom shows the execution of the program with input '6 7 8', resulting in the output 'Triangle is Valid' and 'Scalene Triangle'.

```
9 int main()
10 {
11     if ((a + b > c) && (a + c > b) && (b + c > a)) {
12         printf("Triangle is Valid\n");
13         if (a == b && b == c)
14             printf("Equilateral Triangle");
15         else if (a == b || b == c || a == c)
16             printf("Isosceles Triangle");
17         else if ((a*a + b*b == c*c) || (a*a + c*c == b*b) || (b*b + c*c == a*a))
18             printf("Right Angle Triangle");
19         else
20             printf("Scalene Triangle");
21     }
22     else {
23         printf("Triangle is NOT Valid");
24     }
25     return 0;
26 }
```

Terminal Output:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques7.exe
Enter the three sides of the triangle: 6 7 8
Triangle is Valid
Scalene Triangle
PS C:\Users\DELL\Desktop\100\output>
```

2. WAP to compute the BMI Index of the person and print the BMI values as per the following ranges. You can use the following formula to compute BMI= weight(kgs)/Height(Mts)*Height(Mts).

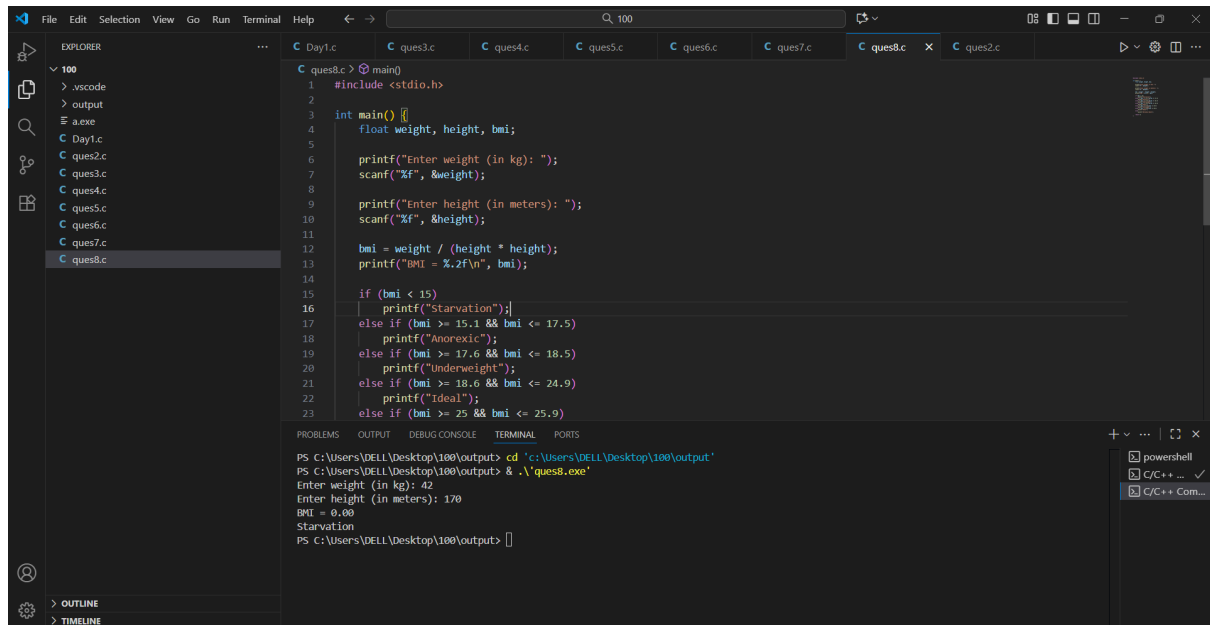
	BMI
starvation	<15
Anorexic	15.1 to 17.5
underweight	17.6 to 18.5
ideal	18.6 to 24.9
overweight	25 to 25.9

obese

30 to 39.9

Morbidity Obese

40.0 above



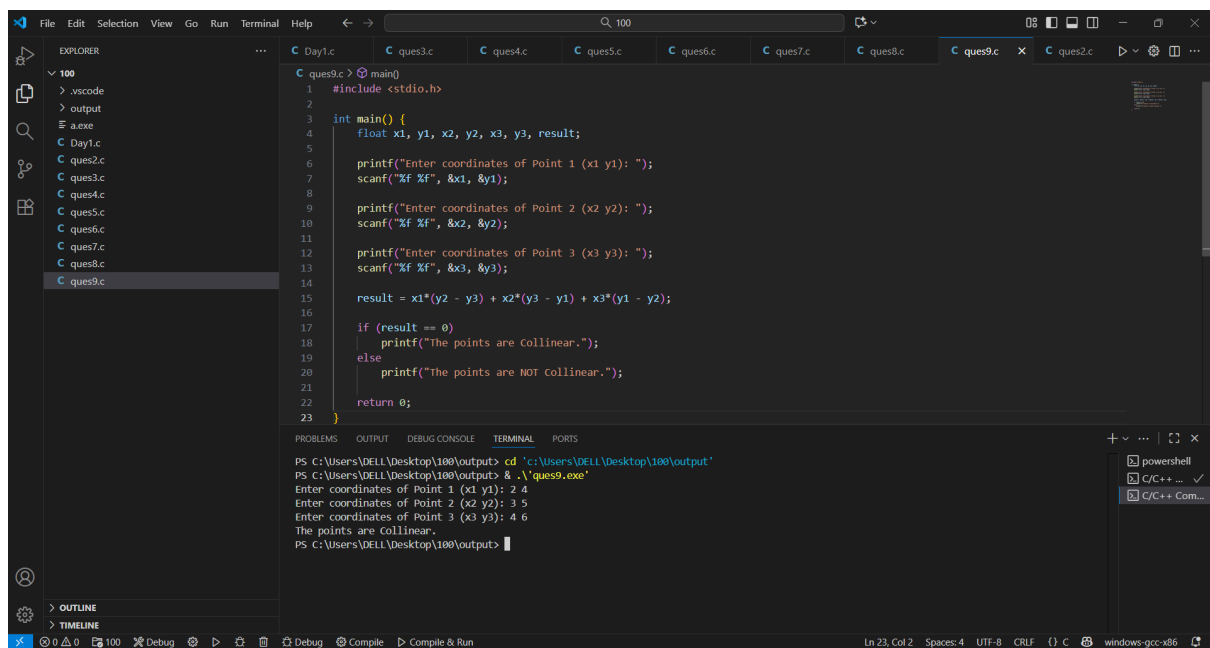
The screenshot shows a Visual Studio Code editor with a C program named `ques8.c` open. The program calculates BMI and categorizes it. The Explorer sidebar shows a project structure with folders `100`, `output`, and `Day1.c`, and several `ques` files. The main editor displays the following C code:

```
1 #include <stdio.h>
2
3 int main() {
4     float weight, height, bmi;
5
6     printf("Enter weight (in kg): ");
7     scanf("%f", &weight);
8
9     printf("Enter height (in meters): ");
10    scanf("%f", &height);
11
12    bmi = weight / (height * height);
13    printf("BMI = %.2f\n", bmi);
14
15    if (bmi < 15)
16        printf("Starvation");
17    else if (bmi >= 15.1 && bmi <= 17.5)
18        printf("Anorexic");
19    else if (bmi >= 17.6 && bmi <= 18.5)
20        printf("Underweight");
21    else if (bmi >= 18.6 && bmi <= 24.9)
22        printf("Ideal");
23    else if (bmi >= 25 && bmi <= 25.9)
```

The Output window shows the execution results:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques8.exe
Enter weight (in kg): 42
Enter height (in meters): 170
BMI = 0.00
Starvation
PS C:\Users\DELL\Desktop\100\output>
```

3. WAP to check if three points (x1,y1), (x2,y2) and (x3,y3) are collinear or not.



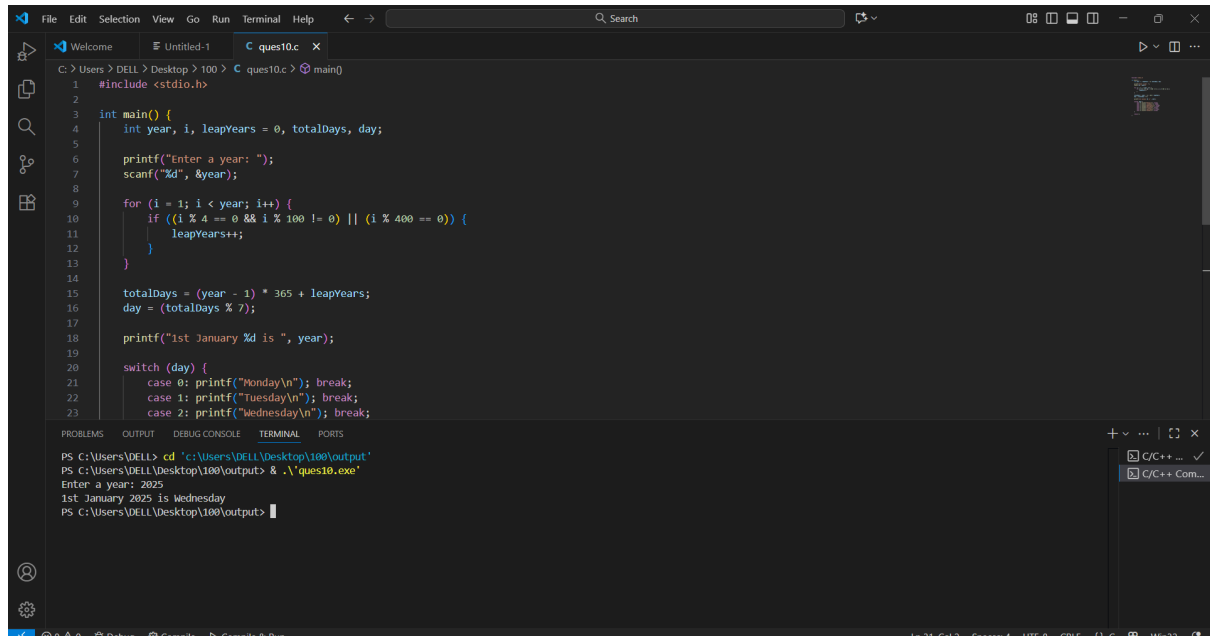
The screenshot shows a Visual Studio Code editor with a C program named `ques9.c` open. The program checks if three points are collinear using the area method. The Explorer sidebar shows a project structure with folders `100`, `output`, and `Day1.c`, and several `ques` files. The main editor displays the following C code:

```
1 #include <stdio.h>
2
3 int main() {
4     float x1, y1, x2, y2, x3, y3, result;
5
6     printf("Enter coordinates of Point 1 (x1 y1): ");
7     scanf("%f %f", &x1, &y1);
8
9     printf("Enter coordinates of Point 2 (x2 y2): ");
10    scanf("%f %f", &x2, &y2);
11
12    printf("Enter coordinates of Point 3 (x3 y3): ");
13    scanf("%f %f", &x3, &y3);
14
15    result = x1*(y2 - y3) + x2*(y3 - y1) + x3*(y1 - y2);
16
17    if (result == 0)
18        printf("The points are Collinear.");
19    else
20        printf("The points are NOT collinear.");
21
22    return 0;
23 }
```

The Output window shows the execution results:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques9.exe
Enter coordinates of Point 1 (x1 y1): 2 4
Enter coordinates of Point 2 (x2 y2): 3 5
Enter coordinates of Point 3 (x3 y3): 4 6
The points are Collinear.
PS C:\Users\DELL\Desktop\100\output>
```

4. According to the gregorian calendar, it was Monday on the date 01/01/01. If Any year is input through the keyboard write a program to find out what is the day on 1st January of this year.



```
File Edit Selection View Go Run Terminal Help
Welcome Untitled-1 ques10.c x
C:\Users\DELL\Desktop\100> ques10.c main()
1 #include <stdio.h>
2
3 int main() {
4     int year, i, leapYears = 0, totalDays, day;
5
6     printf("Enter a year: ");
7     scanf("%d", &year);
8
9     for (i = 1; i < year; i++) {
10         if ((i % 4 == 0 && i % 100 != 0) || (i % 400 == 0)) {
11             leapYears++;
12         }
13     }
14
15     totalDays = (year - 1) * 365 + leapYears;
16     day = (totalDays % 7);
17
18     printf("1st January %d is ", year);
19
20     switch (day) {
21         case 0: printf("Monday\n"); break;
22         case 1: printf("Tuesday\n"); break;
23         case 2: printf("Wednesday\n"); break;
24     }
25 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques10.exe
Enter a year: 2025
1st January 2025 is Wednesday
PS C:\Users\DELL\Desktop\100\output>
```

5. WAP using ternary operator, the user should input the length and breadth of a rectangle, one has to find out which rectangle has the highest perimeter. The minimum number of rectangles should be three.

```

1  #include <stdio.h>
2
3  int main() {
4      int num, positive = 0, negative = 0, zero = 0;
5      char choice;
6
7      do {
8          printf("Enter a number: ");
9          scanf("%d", &num);
10
11         if (num > 0)
12             positive++;
13         else if (num < 0)
14             negative++;
15         else
16             zero++;
17
18         printf("Do you want to continue? (y/n): ");
19         scanf("%c", &choice);
20     } while (choice == 'y' || choice == 'Y');
21
22     printf("\nTotal Positive numbers = %d", positive);
23 }

```

Terminal Output:

```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques11.exe
Enter a number: 5
Do you want to continue? (y/n): 7

Total Positive numbers = 1
Total Negative numbers = 0
Total Zeroes = 0
PS C:\Users\DELL\Desktop\100\output>

```

EXPERIMENT3.2: LOOPS

1. WAP to enter numbers till the user wants. At the end, it should display the count of positive, negative, and Zeroes entered.

```

1  #include <stdio.h>
2
3  int main() {
4      int num;
5      int positive = 0, negative = 0, zero = 0;
6      char choice;
7
8      do {
9          printf("Enter a number: ");
10         scanf("%d", &num);
11
12         if (num > 0)
13             positive++;
14         else if (num < 0)
15             negative++;
16         else
17             zero++;
18
19         printf("Do you want to enter another number? (y/n): ");
20         scanf("%c", &choice);
21     } while (choice == 'y' || choice == 'Y');
22
23 }

```

Terminal Output:

```

PS C:\Users\DELL> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques10.exe
Enter a year: 2025
1st January 2025 is Wednesday
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques12.exe
Enter a number: 4
Do you want to enter another number? (y/n): n

Count of positive numbers: 1
Count of negative numbers: 0
Count of zeroes: 0
PS C:\Users\DELL\Desktop\100\output>

```

2. WAP to print the multiplication table of the number entered by the user. It

should be in the correct formatting. Num * 1 = Num


```
1  #include <stdio.h>
2
3  int main() {
4      int rows = 3;
5      int number = 1;
6      for (int i = 1; i <= rows; i++) {
7          for (int j = 1; j <= i; j++) {
8              printf("%d ", number);
9              number++;
10         }
11         printf("\n");
12     }
13     return 0;
14 }
```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques30.exe

Enter a string: 5

Reversed string: 5

PS C:\Users\DELL\Desktop\100\output>

Compilation successful.

3. WAP to generate the following set of output.

a. 1

 2 3

 4 5 6

```
1  #include <stdio.h>
2
3  int main() {
4      int rows = 3;
5      int number = 1;
6      for (int i = 1; i <= rows; i++) {
7          for (int j = 1; j <= i; j++) {
8              printf("%d ", number);
9              number++;
10         }
11         printf("\n");
12     }
13     return 0;
14 }
```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques14.exe

1

2 3

4 5 6

PS C:\Users\DELL\Desktop\100\output>

Compilation successful.

b.

1

1 1

1 2 1

1 3 3 1

```
1 #include <stdio.h>
2
3
4 int main() {
5     int rows = 4;
6     for (int i = 0; i < rows; i++) {
7         int number = 1;
8         for (int j = 0; j <= i; j++) {
9             printf("%d ", number);
10            number = number * (i - j) / (j + 1);
11        }
12        printf("\\n");
13    }
14    return 0;
15 }
```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques15.exe

1

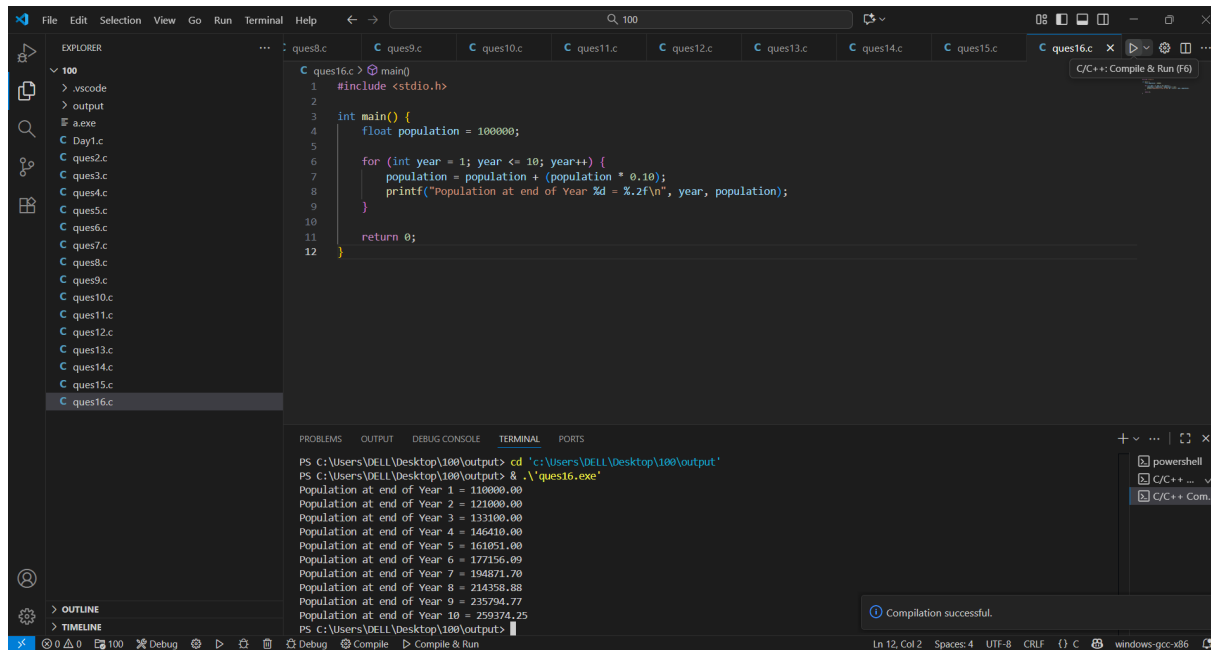
1 1

1 2 1

1 3 3 1

PS C:\Users\DELL\Desktop\100\output>

4. The population of a town is 100000. The population has increased steadily at the rate of 10% per year for the last 10 years. Write a program to determine the population at the end of each year in the last decade.



The screenshot shows a Visual Studio Code editor with a C++ program in a file named `ques16.c`. The program calculates the population of a town over 10 years, starting from 100,000 and increasing by 10% each year. The output is displayed in the terminal window, showing the population at the end of each year from Year 1 to Year 10. The status bar at the bottom indicates that the compilation was successful.

```
1 #include <stdio.h>
2
3 int main() {
4     float population = 100000;
5
6     for (int year = 1; year <= 10; year++) {
7         population = population + (population * 0.10);
8         printf("Population at end of Year %d = %.2f\n", year, population);
9     }
10
11     return 0;
12 }
```

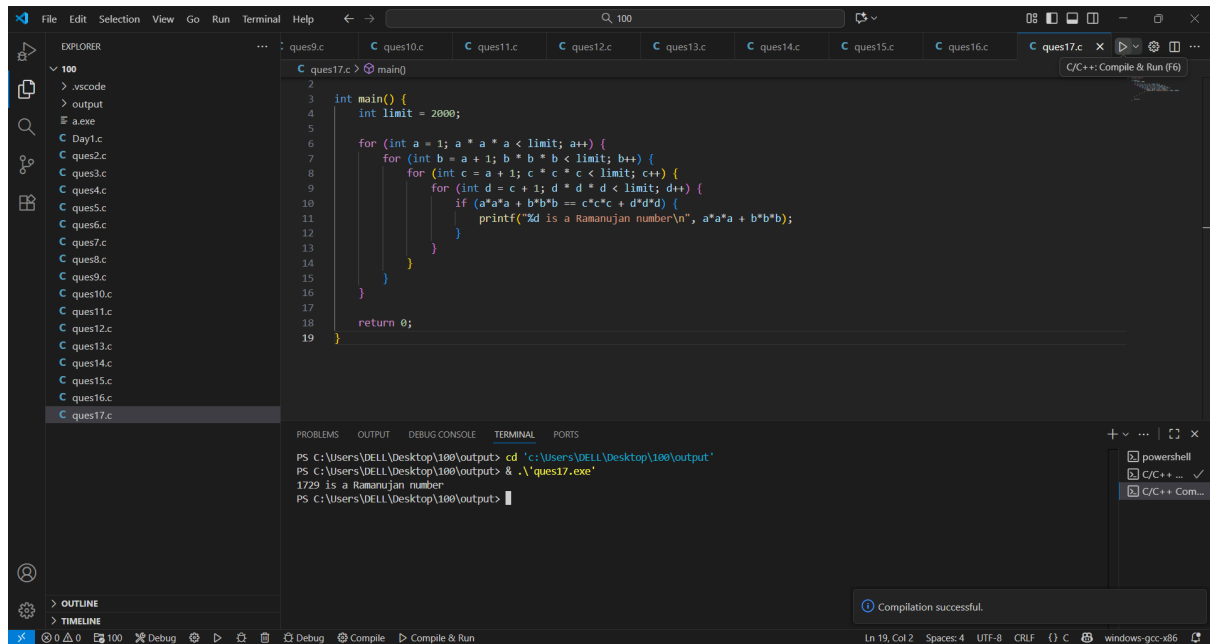
```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> g++ .\ques16.exe
Population at end of Year 1 = 110000.00
Population at end of Year 2 = 121000.00
Population at end of Year 3 = 133100.00
Population at end of Year 4 = 146410.00
Population at end of Year 5 = 161051.00
Population at end of Year 6 = 177156.00
Population at end of Year 7 = 194871.70
Population at end of Year 8 = 214358.88
Population at end of Year 9 = 235794.77
Population at end of Year 10 = 259374.25
PS C:\Users\DELL\Desktop\100\output>
```

Compilation successful.

5. Ramanujan Number is the smallest number that can be expressed as the sum of two cubes in two different ways. WAP to print all such numbers up to a reasonable limit.

Example of Ramanujan number: 1729

$1213 + 1^3$ and $10^3 + 9^3$. for a number $L=20$ (that is limit)



```
1  int main() {
2      int limit = 2000;
3
4      for (int a = 1; a * a * a < limit; a++) {
5          for (int b = a + 1; b * b * b < limit; b++) {
6              for (int c = a + 1; c * c * c < limit; c++) {
7                  for (int d = c + 1; d * d * d < limit; d++) {
8                      if (a*a*a + b*b*b == c*c*c + d*d*d) {
9                          printf("%d is a Ramanujan number\n", a*a*a + b*b*b);
10                     }
11                 }
12             }
13         }
14     }
15     return 0;
16 }
```

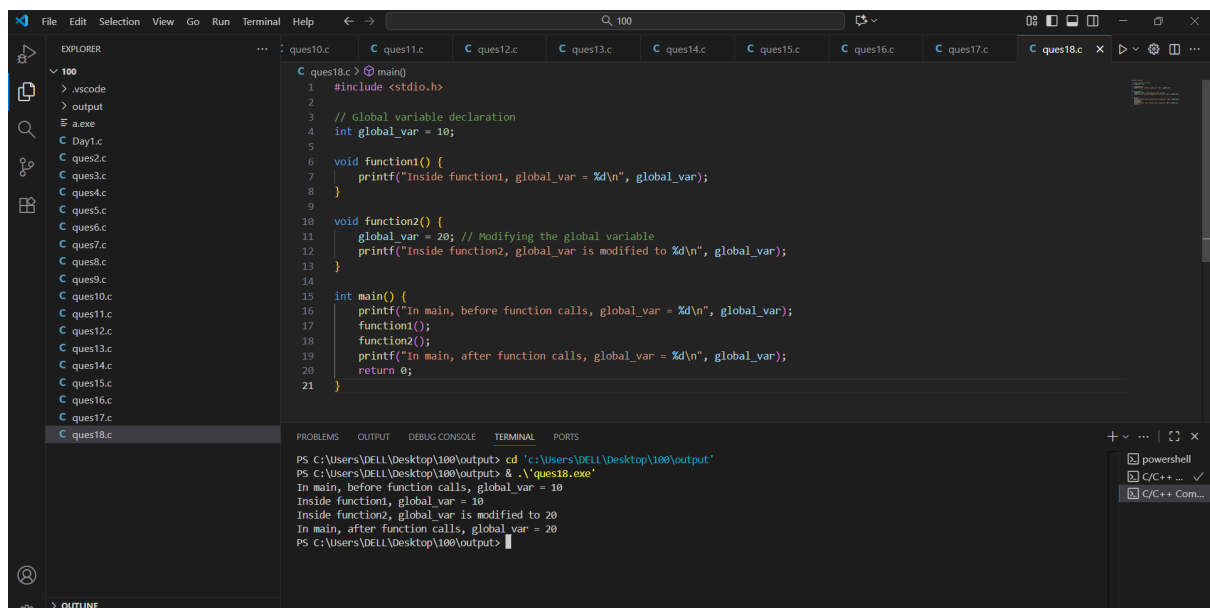
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques17.exe
1729 is a Ramanujan number
PS C:\Users\DELL\Desktop\100\output>
```

Compilation successful.

Experiment 4: Variable and Scope of Variable

1. Declare a global variable outside all functions and use it inside various functions to understand its accessibility.

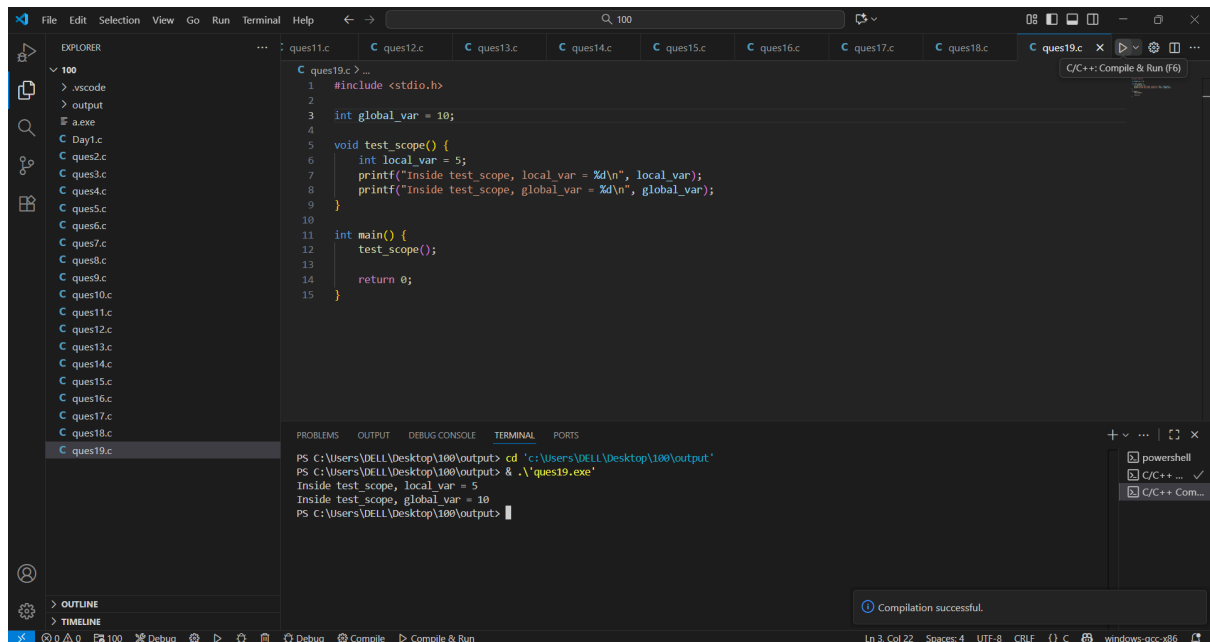


```
1  #include <stdio.h>
2
3  // Global variable declaration
4  int global_var = 10;
5
6  void function1() {
7      printf("Inside function1, global_var = %d\n", global_var);
8  }
9
10 void function2() {
11     global_var = 20; // Modifying the global variable
12     printf("Inside function2, global_var is modified to %d\n", global_var);
13 }
14
15 int main() {
16     printf("In main, before function calls, global_var = %d\n", global_var);
17     function1();
18     function2();
19     printf("In main, after function calls, global_var = %d\n", global_var);
20     return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques18.exe
In main, before function calls, global_var = 10
Inside function1, global_var = 10
Inside function2, global_var is modified to 20
In main, after function calls, global_var = 20
PS C:\Users\DELL\Desktop\100\output>
```

2. Declare a local variable inside a function and try to access it outside the function. Compare this with accessing the global variable from within the function.



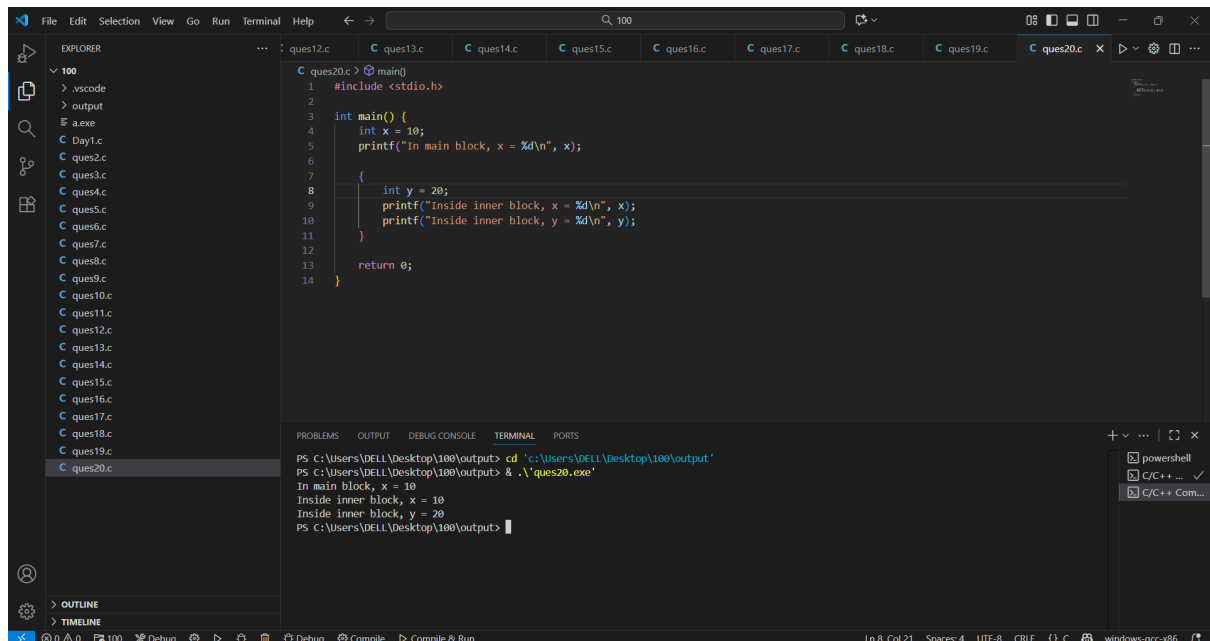
```
1 #include <stdio.h>
2
3 int global_var = 10;
4
5 void test_scope() {
6     int local_var = 5;
7     printf("Inside test_scope, local_var = %d\n", local_var);
8     printf("Inside test_scope, global_var = %d\n", global_var);
9 }
10
11 int main() {
12     test_scope();
13
14     return 0;
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques19.exe
Inside test_scope, local_var = 5
Inside test_scope, global_var = 10
PS C:\Users\DELL\Desktop\100\output>
```

Compilation successful.

3. Declare variables within different code blocks (enclosed by curly braces) and test their accessibility within and outside those blocks.

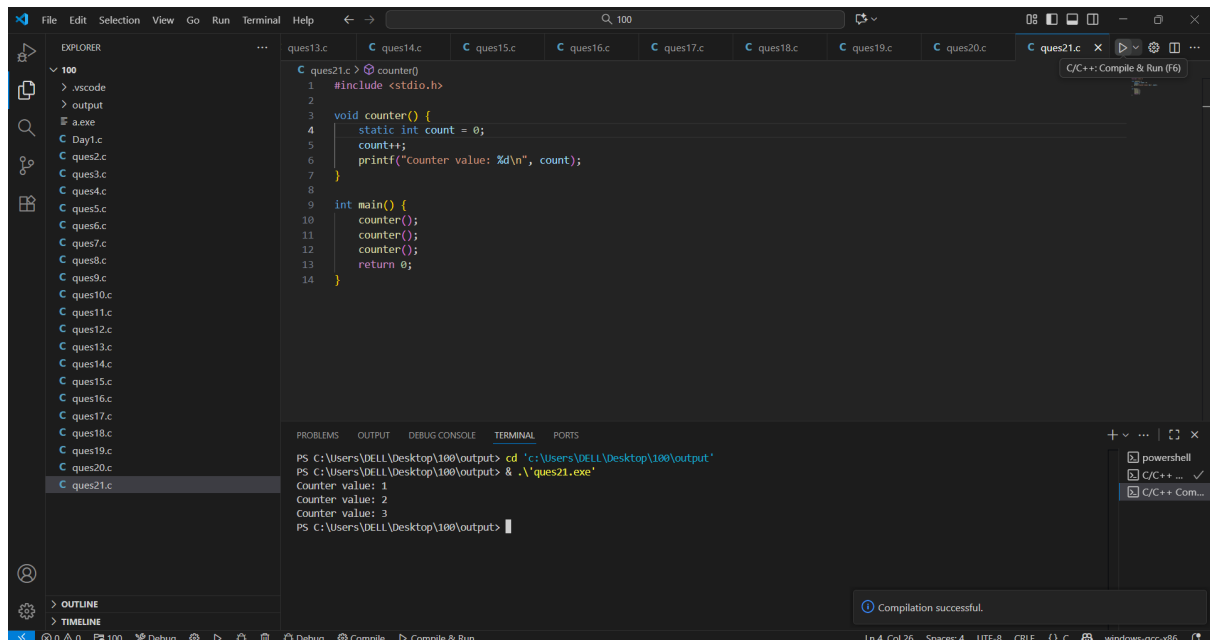


```
1 #include <stdio.h>
2
3 int main() {
4     int x = 10;
5     printf("In main block, x = %d\n", x);
6
7     {
8         int y = 20;
9         printf("Inside inner block, x = %d\n", x);
10        printf("Inside inner block, y = %d\n", y);
11    }
12
13    return 0;
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques20.exe
In main block, x = 10
Inside inner block, x = 10
Inside inner block, y = 20
PS C:\Users\DELL\Desktop\100\output>
```

4. Declare a static local variable inside a function. Observe how its value persists across function calls.



```
1 #include <stdio.h>
2
3 void counter() {
4     static int count = 0;
5     count++;
6     printf("Counter value: %d\n", count);
7 }
8
9 int main() {
10    counter();
11    counter();
12    counter();
13    return 0;
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques21.exe
Counter value: 1
Counter value: 2
Counter value: 3
PS C:\Users\DELL\Desktop\100\output>
```

Compilation successful.

Experiment 5: Array

1. WAP to read a list of integers and store it in a single dimensional array. Write a C program to print the second largest integer in a list of integers.

```
11 for (i = 0; i < n; i++) {
12     scanf("%d", &arr[i]);
13 }
14
15 first = second = arr[0];
16
17 for (i = 1; i < n; i++) {
18     if (arr[i] > first) {
19         second = first;
20         first = arr[i];
21     } else if (arr[i] > second && arr[i] != first) {
22         second = arr[i];
23     }
24 }
25
26 printf("The second largest element is: %d\n", second);
27 return 0;
28 }
```

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques22.exe
Enter number of elements: 3
Enter 3 integers:
2 3 4
The second largest element is: 3
PS C:\Users\DELL\Desktop\100\output>
```

2. WAP to read a list of integers and store it in a single dimensional array. Write a C program to count and display positive, negative, odd, and even numbers in an array.

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, pos = 0, neg = 0, even = 0, odd = 0;
5
6     printf("Enter number of elements: ");
7     scanf("%d", &n);
8
9     int arr[n];
10    printf("Enter %d integers:\n", n);
11    for (i = 0; i < n; i++) {
12        scanf("%d", &arr[i]);
13    }
14
15    for (i = 0; i < n; i++) {
16        if (arr[i] > 0)
17            pos++;
18        else if (arr[i] < 0)
19            neg++;
20
21        if (arr[i] % 2 == 0)
22            even++;
23        else
24            odd++;
25    }
26
27    printf("Positive numbers: %d\n", pos);
28    printf("Negative numbers: %d\n", neg);
29    printf("Even numbers: %d\n", even);
30    printf("Odd numbers: %d\n", odd);
31    return 0;
32 }
```

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques23.exe
Enter number of elements: 3
Enter 3 integers:
3 4 5
Positive numbers: 3
Negative numbers: 0
Even numbers: 1
Odd numbers: 2
PS C:\Users\DELL\Desktop\100\output>
```

3. WAP to read a list of integers and store it in a single dimensional array. Write a C program to find the frequency of a particular number in a list of integers.

```
File Edit Selection View Go Run Terminal Help
C ques24.c
9
10 int arr[n];
11 printf("Enter %d integers:\n", n);
12 for (i = 0; i < n; i++) {
13     scanf("%d", &arr[i]);
14 }
15
16 printf("Enter number to find frequency: ");
17 scanf("%d", &num);
18
19 for (i = 0; i < n; i++) {
20     if (arr[i] == num)
21         count++;
22 }
23
24 printf("Frequency of %d = %d\n", num, count);
25
26 return 0;
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\Desktop\100\output> cd 'C:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques24.exe
Enter number of elements: 4
Enter 4 integers:
3 4 5 6
Enter number to find frequency: 4
Frequency of 4 = 1
PS C:\Users\DELL\Desktop\100\output>

OUTLINE
TIMELINE
Ln 26, Col 2 Spaces: 4 UTF-8 CRLF C windows-gcc-x86
```

4. WAP that reads two matrices A (m x n) and B (p x q) and computes the product A and B. Read matrix A and matrix B in row major order respectively. Print both the input matrices and resultant matrix with suitable headings and output should be in matrix format only. Program must check the compatibility of orders of the matrices for multiplication. Report appropriate message in case of incompatibility.


```

5
6 printf("Enter rows and columns of matrix A: ");
7 scanf("%d%d", &m, &n);
8
9 printf("Enter rows and columns of matrix B: ");
10 scanf("%d%d", &p, &q);
11
12 if (n != p) {
13     printf("Matrix multiplication not possible. Incompatible dimensions.\n");
14     return 0;
15 }
16
17 int A[m][n], B[p][q], C[m][q];
18
19 printf("Enter elements of matrix A:\n");
20 for (i = 0; i < m; i++)
21     for (j = 0; j < n; j++)
22         scanf("%d", &A[i][j]);
23
24 printf("Enter elements of matrix B:\n");
25 for (i = 0; i < p; i++)
26     for (j = 0; j < q; j++)
27         scanf("%d", &B[i][j]);
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Matrix A:

1	2	3
4	5	6

Matrix B:

7	8
9	10
11	12

Resultant Matrix (A * B):

58	64
139	154

Experiment 6: Functions

1. Develop a recursive and non-recursive function FACT(num) to find the factorial of a number, n!, defined by $FACT(n) = 1$, if $n = 0$. Otherwise, $FACT(n) = n * FACT(n-1)$. Using this function, write a C program to compute the binomial coefficient. Tabulate the results for different values of n and r with suitable messages.

```

1 #include <stdio.h>
2
3 long factRec(int n) {
4     if (n == 0)
5         return 1;
6     else
7         return n * factRec(n - 1);
8 }
9
10 long factNonRec(int n) {
11     long f = 1;
12     for (int i = 1; i <= n; i++)
13         f *= i;
14     return f;
15 }
16
17
18 int main() {
19     int n, r;
20     long nCr;
21
22     printf("Enter values for n and r: ");
23     scanf("%d%d", &n, &r);
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques26.exe

Enter values for n and r: 4 5

Invalid input! r should be <= n.

PS C:\Users\DELL\Desktop\100\output>

2. Develop a recursive function GCD (num1, num2) that accepts two integer arguments. Write a C program that invokes this function to find the greatest common divisor of two given integers.

```
1 #include <stdio.h>
2
3 int GCD(int a, int b) {
4     if (b == 0)
5         return a;
6     else
7         return GCD(b, a % b);
8 }
9
10
11 int main() {
12     int num1, num2;
13     printf("Enter two integers: ");
14     scanf("%d%d", &num1, &num2);
15
16     printf("GCD of %d and %d is %d\n", num1, num2, GCD(num1, num2));
17     return 0;
18 }
```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> .\ques27.exe

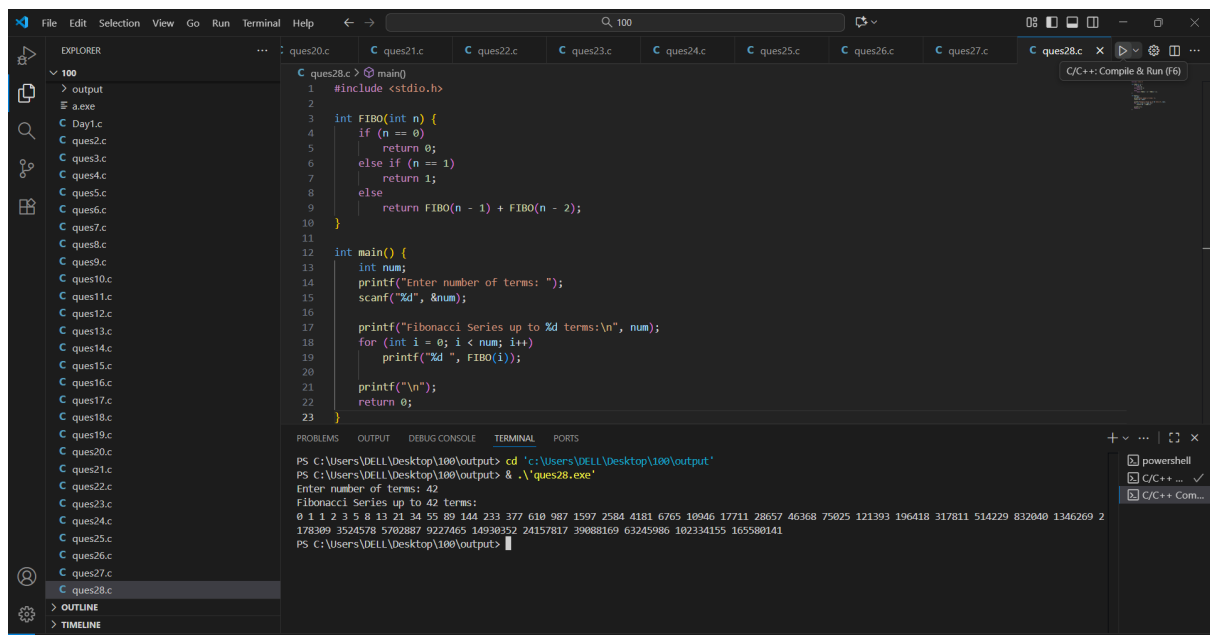
Enter two integers: 3 5

GCD of 3 and 5 is 1

PS C:\Users\DELL\Desktop\100\output> |

Compilation successful.

3. Develop a recursive function FIBO (num) that accepts an integer argument. Write a C program that invokes this function to generate the Fibonacci sequence up to num.



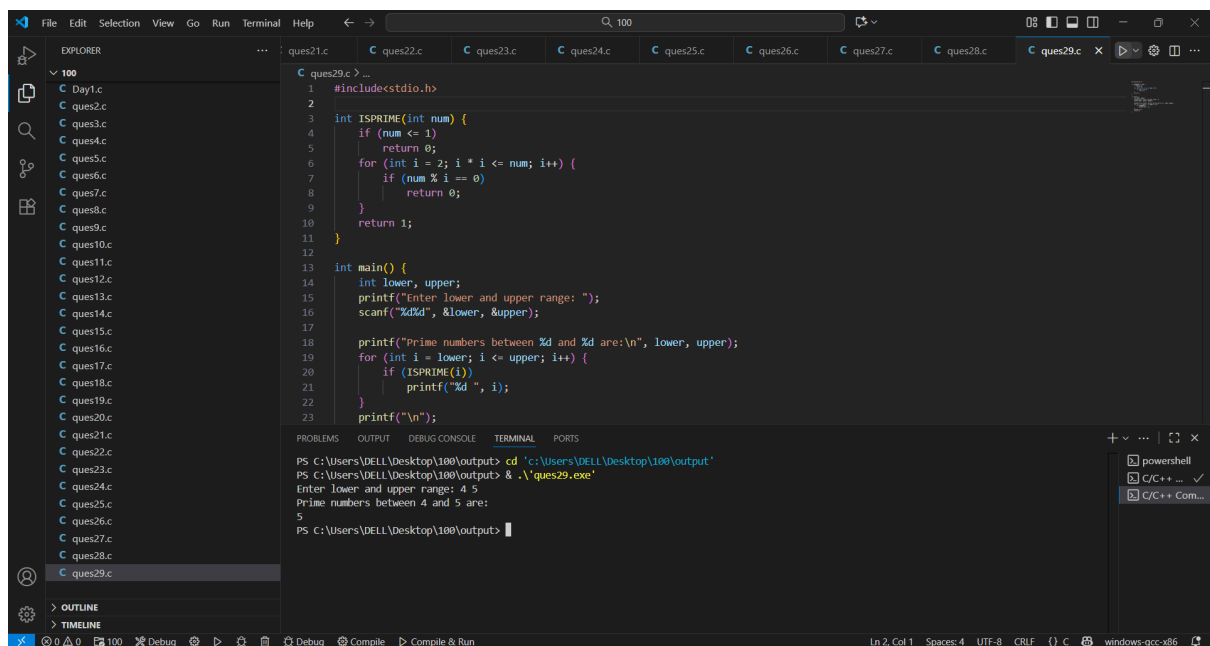
The screenshot shows a Visual Studio Code editor with a C program named `ques28.c`. The program defines a recursive function `FIBO` and a `main` function. The `main` function prompts the user to enter the number of terms, which is 42. It then prints the Fibonacci series up to 42 terms. The output in the terminal shows the sequence: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368 75825 121393 196418 317811 514229 832040 1346269 2178309 3524578 5702887 9227465 14930352 24157817 39088169 63245986 102334155 165580141.

```
1 #include <stdio.h>
2
3 int FIBO(int n) {
4     if (n == 0)
5         return 0;
6     else if (n == 1)
7         return 1;
8     else
9         return FIBO(n - 1) + FIBO(n - 2);
10 }
11
12 int main() {
13     int num;
14     printf("Enter number of terms: ");
15     scanf("%d", &num);
16
17     printf("Fibonacci Series up to %d terms:\n", num);
18     for (int i = 0; i < num; i++)
19         printf("%d ", FIBO(i));
20
21     printf("\n");
22     return 0;
23 }
```

Terminal Output:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques28.exe
Enter number of terms: 42
Fibonacci Series up to 42 terms:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368 75825 121393 196418 317811 514229 832040 1346269 2178309 3524578 5702887 9227465 14930352 24157817 39088169 63245986 102334155 165580141
PS C:\Users\DELL\Desktop\100\output>
```

4. Develop a C function ISPRIME (num) that accepts an integer argument and returns 1 if the argument is prime, a 0 otherwise. Write a C program that invokes this function to generate prime numbers between the given ranges.



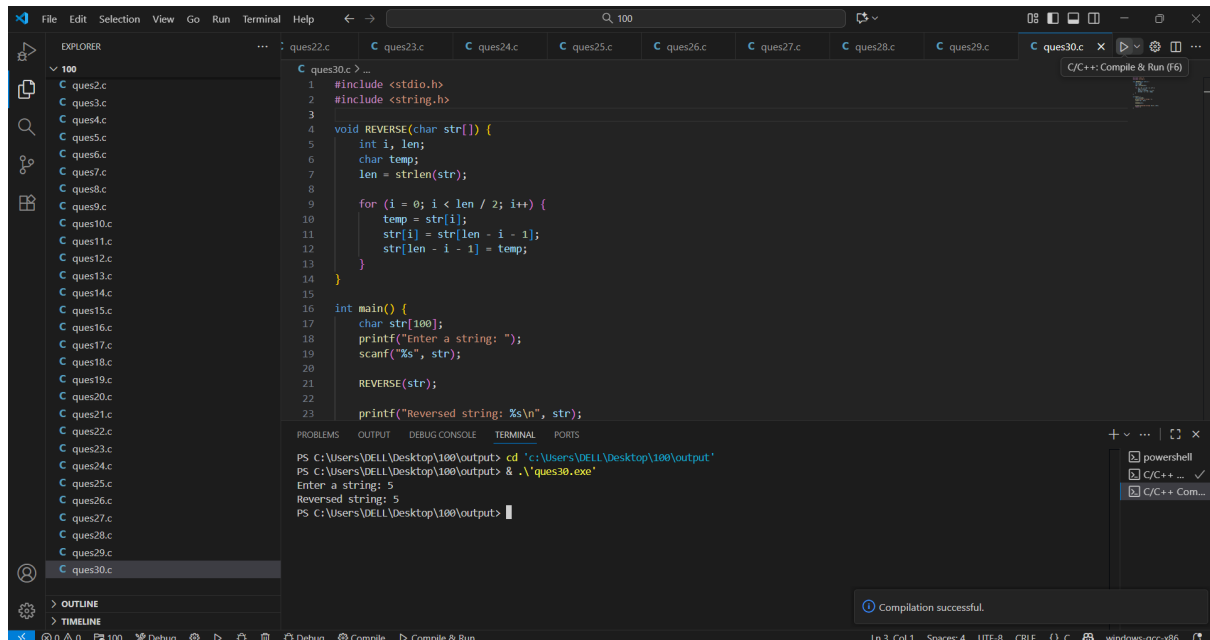
The screenshot shows a Visual Studio Code editor with a C program named `ques29.c`. The program defines a function `ISPRIME` and a `main` function. The `main` function prompts the user to enter a lower and upper range, which are 4 and 5. It then prints the prime numbers between 4 and 5, which are 5.

```
1 #include <stdio.h>
2
3 int ISPRIME(int num) {
4     if (num <= 1)
5         return 0;
6     for (int i = 2; i * i <= num; i++) {
7         if (num % i == 0)
8             return 0;
9     }
10    return 1;
11 }
12
13 int main() {
14     int lower, upper;
15     printf("Enter lower and upper range: ");
16     scanf("%d%d", &lower, &upper);
17
18     printf("Prime numbers between %d and %d are:\n", lower, upper);
19     for (int i = lower; i <= upper; i++) {
20         if (ISPRIME(i))
21             printf("%d ", i);
22     }
23     printf("\n");
24 }
```

Terminal Output:

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques29.exe
Enter lower and upper range: 4 5
Prime numbers between 4 and 5 are:
5
PS C:\Users\DELL\Desktop\100\output>
```

5. Develop a function REVERSE (str) that accepts a string argument. Write a C program that invokes this function to find the reverse of a given string.



```
1 #include <stdio.h>
2 #include <string.h>
3
4 void REVERSE(char str[]) {
5     int i, len;
6     char temp;
7     len = strlen(str);
8
9     for (i = 0; i < len / 2; i++) {
10        temp = str[i];
11        str[i] = str[len - i - 1];
12        str[len - i - 1] = temp;
13    }
14 }
15
16 int main() {
17     char str[100];
18     printf("Enter a string: ");
19     scanf("%s", str);
20
21     REVERSE(str);
22
23     printf("Reversed string: %s\n", str);
24 }
```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques30.exe

Enter a string: 5

Reversed string: 5

PS C:\Users\DELL\Desktop\100\output>

Compilation successful.

Experiment 7: Structures and Union

1. Write a C program that uses functions to perform the following operations:

- Reading a complex number.
- Writing a complex number.
- Addition and subtraction of two complex numbers

Note: represent complex number using a structure.

```
14
15 int main() {
16     struct complex a, b, add, sub;
17
18     a = read();
19     b = read();
20
21     add.r = a.r + b.r;
22     add.i = a.i + b.i;
23
24     sub.r = a.r - b.r;
25     sub.i = a.i - b.i;
26
27     print(add);
28     print(sub);
29
30     return 0;
31 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\100> cd 'C:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques31.exe

10 5

5 7

15.0 + 12.0i

5.0 + -2.0i

PS C:\Users\DELL\Desktop\100\output> |

2. Write a C program to compute the monthly pay of 100 employees using each employee_s name, basic pay. The DA is computed as 52% of the basic pay.

Gross-salary (basic pay + DA). Print the employees name and gross salary.

```
1 #include <stdio.h>
2
3 struct emp {
4     char name[30];
5     float basic, gross;
6 };
7
8 int main() {
9     struct emp e[100];
10    int i;
11
12    for(i = 0; i < 100; i++) {
13        scanf("%s %f", e[i].name, &e[i].basic);
14        e[i].gross = e[i].basic + (0.52 * e[i].basic);
15    }
16
17    for(i = 0; i < 100; i++)
18        printf("%s %.2f\n", e[i].name, e[i].gross);
19
20    return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

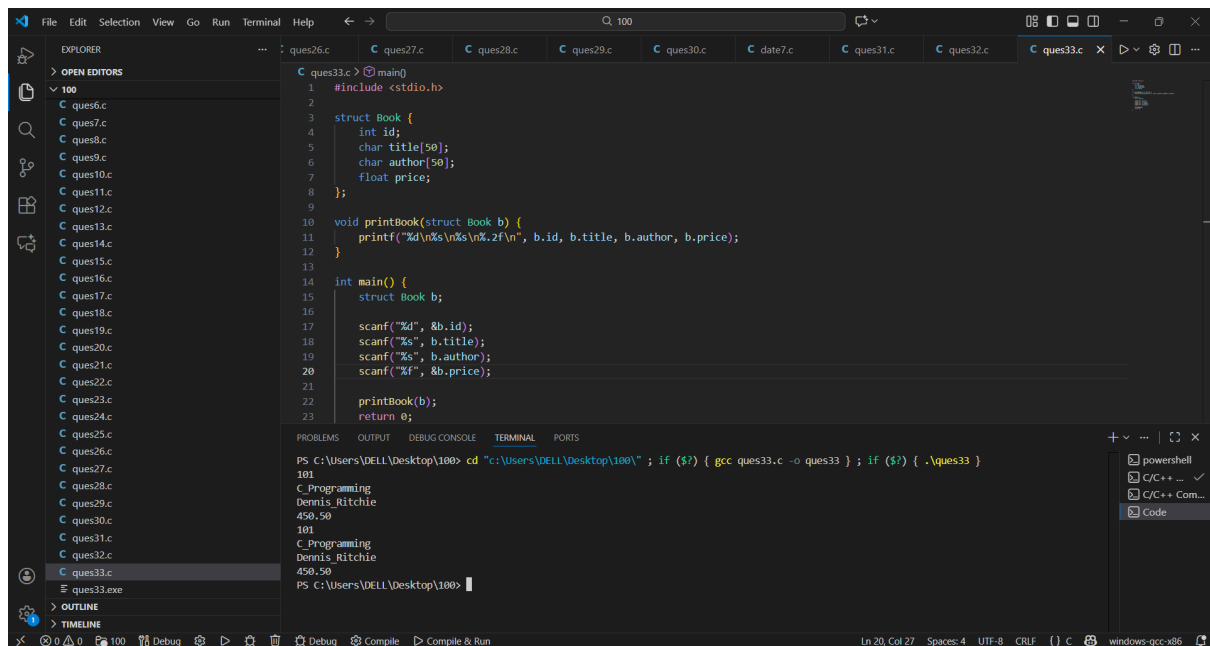
PS C:\Users\DELL\Desktop\100\output> cd 'C:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques32.exe

|

Compilation successful.

3. Create a Book structure containing book_id, title, author name and price. Write a C program to pass a structure as a function argument and print the book details.



```
1 #include <stdio.h>
2
3 struct Book {
4     int id;
5     char title[50];
6     char author[50];
7     float price;
8 };
9
10 void printBook(struct Book b) {
11     printf("%d\n%s\n%s\n%.2f\n", b.id, b.title, b.author, b.price);
12 }
13
14 int main() {
15     struct Book b;
16
17     scanf("%d", &b.id);
18     scanf("%s", b.title);
19     scanf("%s", b.author);
20     scanf("%f", &b.price);
21
22     printBook(b);
23     return 0;
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\DELL\Desktop\100> cd "c:\Users\DELL\Desktop\100\" ; if ($?) { gcc ques33.c -o ques33 } ; if ($?) { .\ques33 }
101
C Programming
Dennis Ritchie
450.50
101
C Programming
Dennis Ritchie
450.50
PS C:\Users\DELL\Desktop\100>
```

4. Create a union containing 6 strings: name, home_address, hostel_address, city, state and zip. Write a C program to display your present address.

```

1  #include <stdio.h>
2
3  union Address {
4      char name[50];
5      char home[50];
6      char hostel[50];
7      char city[50];
8      char state[50];
9      char zip[20];
10 };
11
12 int main() {
13     union Address a;
14
15     printf("Enter present address: ");
16     gets(a.hostel); // or a.home depending on your present address
17
18     printf("Present Address: %s", a.hostel);
19
20     return 0;
21 }

```

```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques34.exe
Enter present address: dehradun
Present Address: dehradun
PS C:\Users\DELL\Desktop\100\output>

```

Experiment 8: Pointers

1. Declare different types of pointers (int, float, char) and initialize them with the addresses of variables. Print the values of both the pointers and the variables they point to.

```

1  #include <stdio.h>
2
3  int main() {
4      int a = 10;
5      float b = 5.5;
6      char c = 'X';
7
8      int *p1 = &a;
9      float *p2 = &b;
10     char *p3 = &c;
11
12     printf("%p %d\n", p1, *p1);
13     printf("%p %.1f\n", p2, *p2);
14     printf("%p %c\n", p3, *p3);
15
16     return 0;
17 }

```

```

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques35.exe
0061FF10 10
0061FF0C 5.5
0061FF0B X
PS C:\Users\DELL\Desktop\100\output>

```

2. Perform pointer arithmetic (increment and decrement) on pointers of different data types. Observe how the memory addresses change and the effects on data access.

```

1 #include <stdio.h>
2
3 int main() {
4     int a = 10, *p1 = &a;
5     float b = 5.5, *p2 = &b;
6     char c = 'X', *p3 = &c;
7
8     printf("Int pointer: %p\n", p1);
9     p1++;
10    printf("p1++ = %p\n", p1);
11    p1--;
12    printf("p1-- = %p\n\n", p1);
13
14    printf("float pointer: %p\n", p2);
15    p2++;
16    printf("p2++ = %p\n", p2);
17    p2--;
18    printf("p2-- = %p\n\n", p2);
19
20    printf("Char pointer: %p\n", p3);
21    p3++;
22    printf("p3++ = %p\n", p3);
23    p3--;
24
25    return 0;
26 }

```

Output:

```

PS C:\Users\DELL\Desktop\100\output> & .\ques36.exe
Int pointer: 0061FF10
p1++ = 0061FF14
p1-- = 0061FF10

Float pointer: 0061FF0C
p2++ = 0061FF10
p2-- = 0061FF0C

Char pointer: 0061FF0B
p3++ = 0061FF0C
p3-- = 0061FF0B
PS C:\Users\DELL\Desktop\100\output>

```

3. Write a function that accepts pointers as parameters. Pass variables by reference using pointers and modify their values within the function.

```

1 #include <stdio.h>
2
3 void change(int *x, float *y) {
4     *x = *x + 10; // modify integer
5     *y = *y * 2; // modify float
6 }
7
8 int main() {
9     int a = 5;
10    float b = 3.5;
11
12    change(&a, &b);
13
14    printf("%d\n", a);
15    printf("%.1f\n", b);
16
17    return 0;
18 }

```

Output:

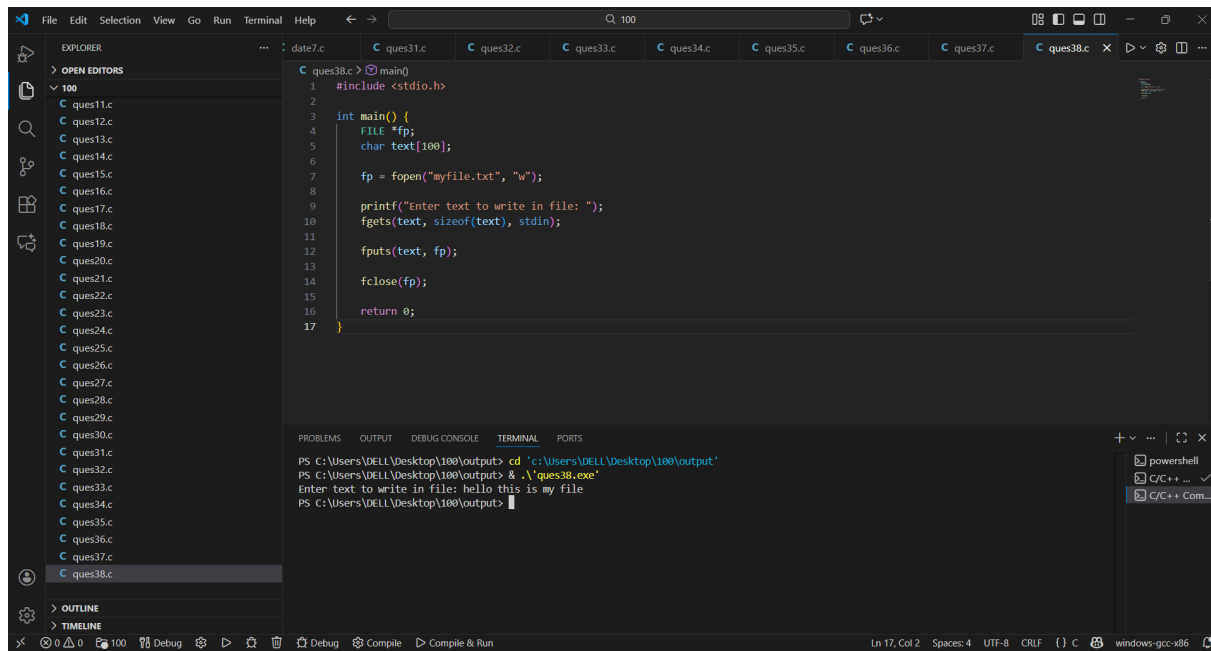
```

PS C:\Users\DELL\Desktop\100> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> & .\ques37.exe
15
7.0
PS C:\Users\DELL\Desktop\100\output>

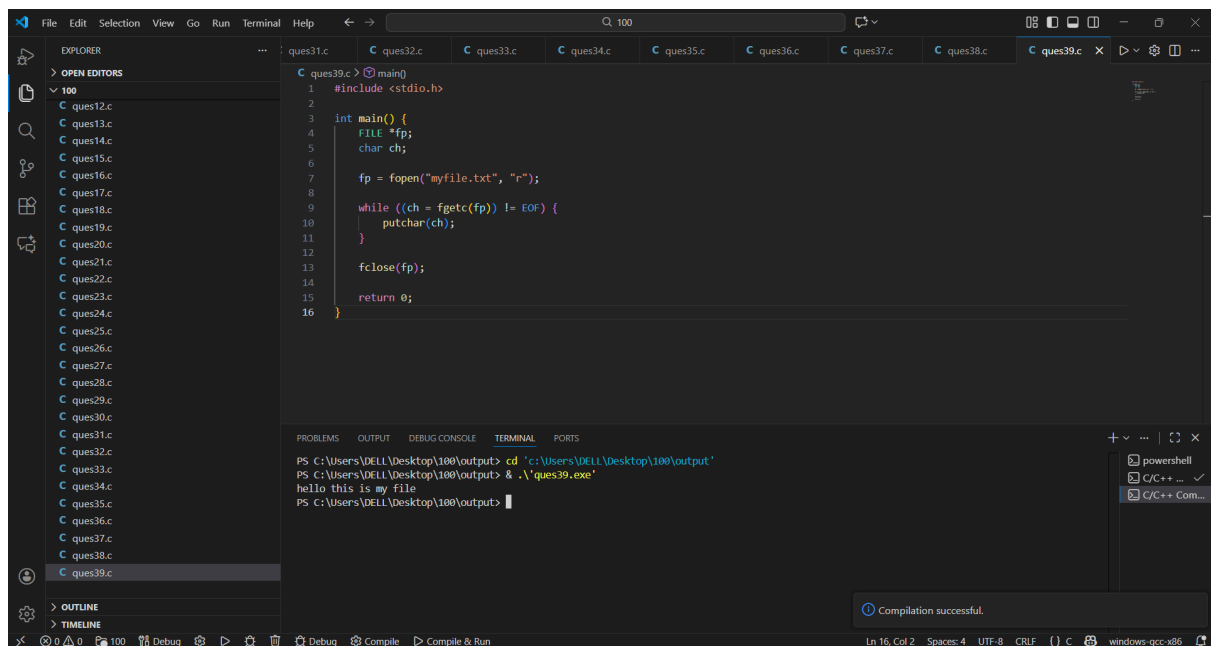
```

Experiment 9: file Handling in C

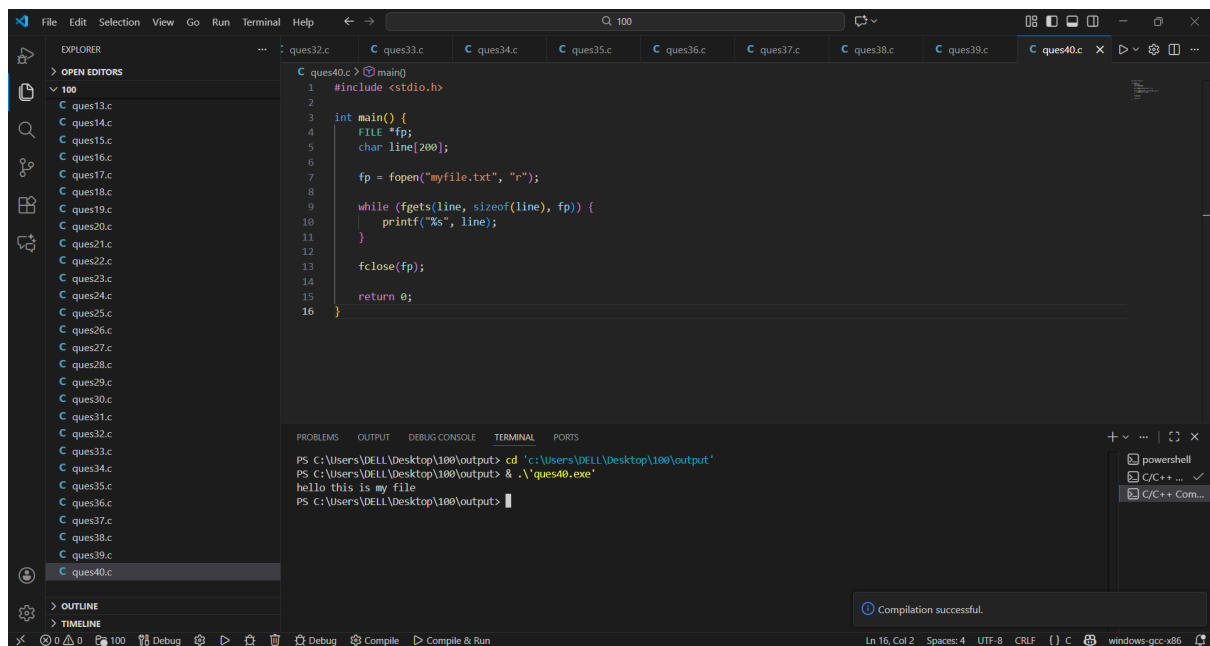
1. Write a program to create a new file and write text into it.



2. Open an existing file and read its content character by character, and then close the file.

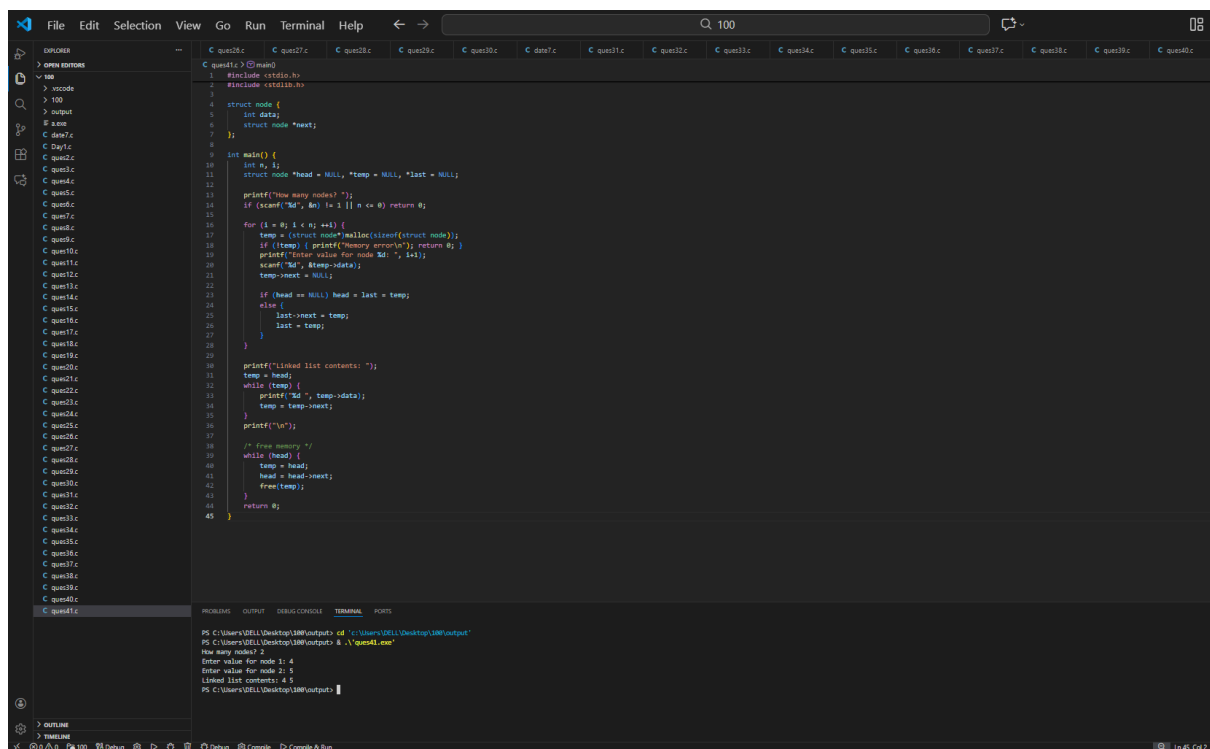


3. Open a file, read its content line by line, and display each line on the console.

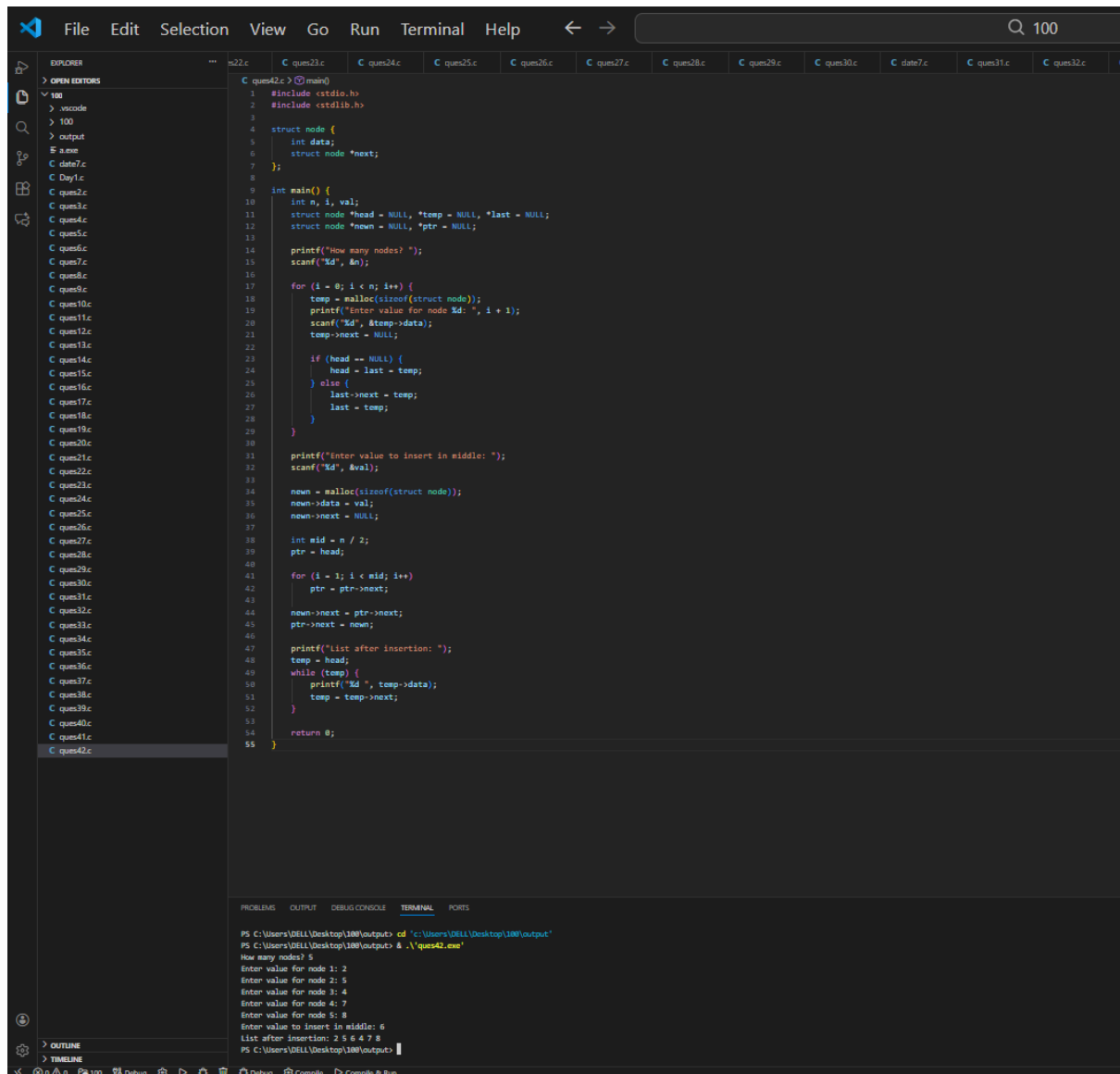


Experiment 10: Dynamic Memory Allocation

1. Write a program to create a simple linked list in C using pointer and structure.



2. Write a program to insert item in middle of the linked list.



Experiment 11: Bitwise Operator

1. Write a program to apply bitwise OR, AND and NOT operators on bit level.

```
1 #include <stdio.h>
2
3 int main() {
4     int a, b;
5
6     printf("Enter two numbers: ");
7     scanf("%d %d", &a, &b);
8
9     printf("a & b = %d\n", a & b);
10    printf("a | b = %d\n", a | b);
11    printf("~a = %d\n", ~a);
12
13    return 0;
14 }
```

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques43.exe
Enter two numbers: 4 5
a & b = 4
a | b = 5
~a = -5
PS C:\Users\DELL\Desktop\100\output>
```

Compilation successful.

2. Write a program to apply left shift and right shift operator.

```
1 #include <stdio.h>
2
3 int main() {
4     int a;
5
6     printf("Enter a number: ");
7     scanf("%d", &a);
8
9     printf("Left shift (a << 1) = %d\n", a << 1);
10    printf("Right shift (a >> 1) = %d\n", a >> 1);
11
12    return 0;
13 }
```

```
PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'
PS C:\Users\DELL\Desktop\100\output> .\ques44.exe
Enter a number: 4
Left shift (a << 1) = 8
Right shift (a >> 1) = 2
PS C:\Users\DELL\Desktop\100\output>
```

Compilation successful.

Experiment 12: Preprocessor and Directives in C

1. Write a program to define some constant variable in preprocessor.

```
1 #include <stdio.h>
2 #define PI 3.14
3 #define MAX 100
4
5 int main() {
6     printf("PI = %f\n", PI);
7     printf("MAX = %d\n", MAX);
8     return 0;
9 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques45.exe

PI = 3.140000

MAX = 100

PS C:\Users\DELL\Desktop\100\output> |

Compilation successful.

2. Write a program to define a function in directives.

```
1 #include <stdio.h>
2 #define SQUARE(x) ((x)*(x))
3
4 int main() {
5     int n;
6     printf("Enter a number: ");
7     scanf("%d", &n);
8     printf("Square = %d\n", SQUARE(n));
9     return 0;
10 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\DELL\Desktop\100\output> cd 'c:\Users\DELL\Desktop\100\output'

PS C:\Users\DELL\Desktop\100\output> & .\ques46.exe

Enter a number: 4

Square = 16

PS C:\Users\DELL\Desktop\100\output> |

Compilation successful.